

READER-FRIENDLY PDF EDITION

Ferryte: The Complete Guide

"I know nothing about AI" to "I understand Ferryte better than anyone"

SOURCE LINES 948	DIAGRAMS 12	TABLES 6
---------------------	----------------	-------------



Preservation note: this PDF keeps the original guide content intact and adds print styling for readability. Mermaid code blocks are rendered as diagrams, while the source Markdown file remains unchanged.

Ferryte: The Complete Guide

From “I know nothing about AI” to “I understand Ferryte better than anyone”

How to read this book

This guide is built so that someone with **zero** technical background can finish it and understand Ferryte deeply — the problem, the product, the technology, the business, and the marketing.

Two rules I follow everywhere in this book:

1. **Plain words first.** Whenever a technical term shows up, I explain it in plain English immediately after, in a quoted box like this:

In plain words: this is where the simple explanation lives. If a sentence above confused you, the answer is in the box right below it.

2. **A picture for every hard idea.** Diagrams use a format called *Mermaid*. If you’re reading this on GitHub or in a markdown viewer, they render as real flowcharts. If you see raw text in a code box instead, just read it top-to-bottom like a recipe — arrows (`-->`) mean “leads to.”

Take your time. There’s a **Glossary** at the very end — every bolded term is defined there too.

Table of contents

- **Part 1 — The 60-second version**
- **Part 2 — Foundations: how AI agents and memory actually work** (start here if AI is new to you)
- **Part 3 — The Problem: why “delete” doesn’t really delete**
- **Part 4 — The Solution: what Ferryte does**

- **Part 5 — Under the hood: how Ferryte is built**
 - **Part 6 — The four tests (scenarios) explained**
 - **Part 7 — The Benchmark: “The Forgetting Report”**
 - **Part 8 — Positioning & marketing: how we talk about Ferryte**
 - **Part 9 — The business: free Core, paid Cloud, paid Enterprise**
 - **Part 10 — The roadmap: what we plan to build**
 - **Part 11 — The honest weaknesses (read this, it matters)**
 - **Part 12 — Glossary of every term**
-

Part 1 — The 60-second version

Ferryte is a tool that checks one thing: **when you tell an AI system to forget something, did it actually forget?**

It turns out the answer is usually **no** — and almost nobody is checking. When an app deletes a piece of data, the AI has often already copied that data into other places (summaries, notes, search indexes). Deleting the original doesn't remove the copies. So the AI keeps "remembering" things it was told to forget. That can leak a customer's private data, break privacy law, or expose one company's data to another.

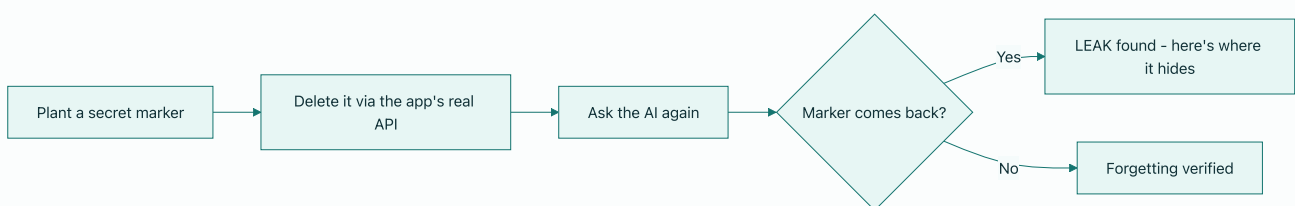
Ferryte:

1. Plants a secret "marker" in the AI's memory.
2. Tells the AI to delete it (using the app's real delete button).
3. Asks the AI about it again.
4. If the marker comes back, that's a **leak**, and Ferryte shows you exactly where it's hiding.

Then a feature called the **cascade** can clean up every hidden copy automatically.

That's the whole idea. The rest of this book explains *why* this happens, *how* Ferryte does it, and *why it's a business*.

DIAGRAM 1



Part 2 — Foundations: how AI agents and memory actually work

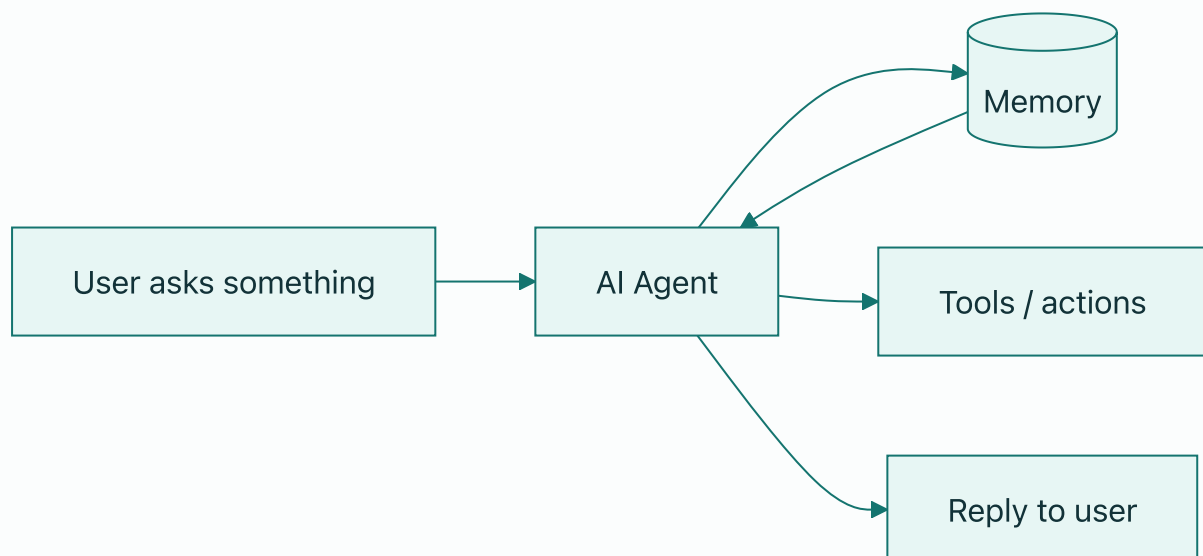
If you already know what an LLM, embedding, and vector database are, skip to Part 3. Otherwise, read this part slowly — everything later depends on it.

2.1 What is an “AI agent”?

A normal program does exactly what it’s told, step by step. An **AI agent** is a program built around a **large language model (LLM)** — software that can read and write human language — that can *decide* what to do, take actions, and remember things between conversations.

In plain words: a chatbot that can also *do stuff* (look things up, call other tools, take actions) and *remember* you between chats is an “agent.” Think of a very capable virtual assistant.

DIAGRAM 2



The key new ingredient versus an old chatbot is the box labeled **Memory**. That box is the entire reason Ferryte exists.

2.2 What is an LLM?

LLM stands for *Large Language Model*. It's the "brain" — a piece of software trained on huge amounts of text that can continue, summarize, translate, or answer in natural language.

Examples: GPT, Claude, Gemini.

In plain words: the LLM is the part that "talks" and "thinks." On its own it has no memory of you — every conversation starts blank unless something *feeds it* your past. That something is the memory system.

2.3 What is "memory" for an AI agent?

Because an LLM forgets everything between messages, agents bolt on a **memory system** — a place to store facts about you and your past conversations, and pull them back later. When you tell an assistant "I'm allergic to peanuts," a good agent stores that and remembers it next week.

There are a few common ways memory is stored. You need to understand three words:

Embeddings

An **embedding** is a way to turn a piece of text into a list of numbers that captures its *meaning*. Texts with similar meaning get similar numbers.

In plain words: imagine giving every sentence a "GPS coordinate" based on what it means, so that "I love dogs" and "puppies are great" end up near each other on the map. That coordinate is the embedding.

Vector database (vector store)

A **vector database** stores those number-lists (called *vectors*) and can quickly find the ones "nearest" to a question. That's how the agent finds relevant memories.

In plain words: it's a filing cabinet that files notes by *meaning* instead of by date or name, so when you ask a question it pulls the notes that *mean* something similar. Examples you'll see in this book: **pgvector**, **Chroma**, **Qdrant**.

Summary / extracted facts (this is the important one)

To save space and work better, agents don't just store your raw words. They use the LLM to **summarize** ("the user is a vegan who lives in Berlin and dislikes spicy food") and to **extract**

facts ("allergy: peanuts"). These summaries and facts are *new pieces of text the AI wrote in its own words*.

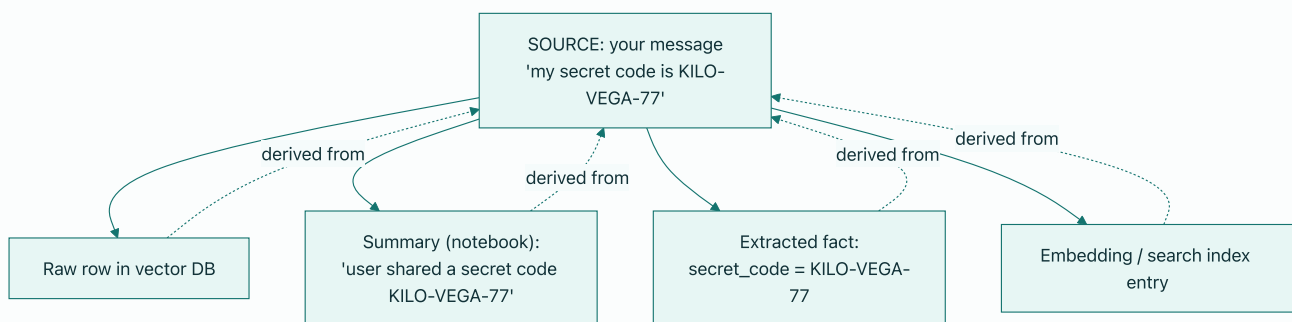
In plain words: the AI keeps a **personal notebook** where it rewrites what you told it. Your original message is one thing; the AI's notebook entry about it is a *separate copy*, in different words.

Remember the notebook. The notebook is the villain of this whole story.

Derived memory

Anything the AI *created from* your original data — a summary, an extracted fact, an embedding, a knowledge-graph node — is called **derived memory** (or a *derived artifact*). Your original message is the **source**. Everything the AI made from it is *derived*.

DIAGRAM 3



In plain words: one thing you said becomes *many* copies inside the AI — the original, the notebook summary, the bullet-point fact, and the search-index entry.

2.4 Multi-tenant: one app, many customers

A **multi-tenant** system is one app serving many separate customers ("tenants"), each of whom must only ever see *their own* data. A customer-support AI used by 500 different companies is multi-tenant: Company A must never see Company B's data.

In plain words: like an apartment building. Everyone shares the building (the app), but you must never be able to walk into someone else's apartment (their data). Multi-tenant AI memory is where leaks get *really* dangerous.

You now have all the vocabulary you need. On to the problem.

Part 3 — The Problem: why “delete” doesn’t really delete

3.1 The core failure, in one sentence

You tell an AI to forget something. It says “done.” It hasn’t.

When an app deletes data, it usually deletes only the **one thing it knows about** — typically the raw row in the database. But by then the AI has already copied that data into the **notebook** (summaries), the **fact list**, and the **search index**. Those copies don’t get deleted. So the AI keeps answering from them.

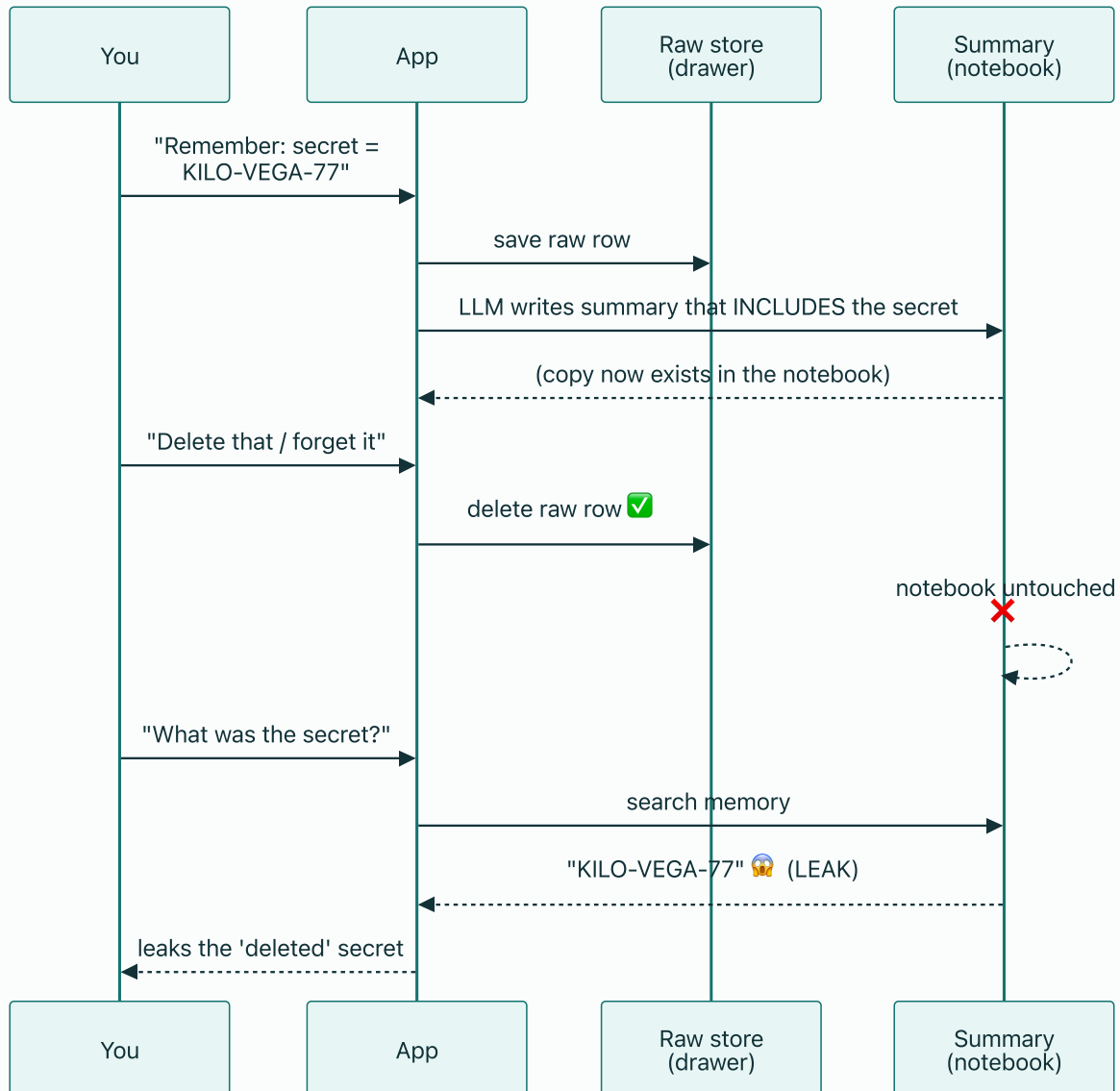
3.2 The notebook analogy (the whole problem in a picture)

Imagine you hand an assistant a letter with a secret on it. The assistant:

- files the letter in a drawer (the raw row),
- copies the secret into a shared team notebook (the summary),
- writes it on a sticky note (the extracted fact).

Later you say “shred that letter.” The assistant shreds the **letter** — but the **notebook** and the **sticky note** still have the secret. Ask again, and the assistant happily reads it back from the notebook.

DIAGRAM 4



In plain words: deleting the original doesn't un-write the copies the AI already made. The AI keeps "remembering" the deleted thing from its notebook.

3.3 This is not a rare bug — the vendors document it themselves

This isn't a weird edge case. The companies that *build* agent memory say so in their own manuals:

- **AWS Bedrock AgentCore** (Amazon's agent-memory service) literally writes: *"Deleting an event doesn't remove the structured information derived out of it from the long term memory."*

- **Zep** (another memory product): deleting an episode doesn't regenerate the shared summaries that already absorbed it.
- **Mem0** and generic vector stores: you delete the row, but copies already pulled into the agent's working set keep coming back.

In plain words: the people who make these memory systems openly admit, in their documentation, that "delete" doesn't delete the copies. Almost no company building on top of them checks whether this is hurting them.

3.4 Why it's *architectural*, not a quick fix

It's tempting to think "they'll just patch it." They won't easily, because the copying is *the whole point* of the system. Summaries and extracted facts are what make agent memory fast and useful. The leak is a side-effect of a feature, not a typo in the code.

In plain words: the AI copies your data *on purpose*, because that's what makes it smart. The leak is baked into how the technology works — so it needs a dedicated tool to catch and clean it, not a one-line fix.

3.5 Why anyone should care — the real-world damage

This isn't academic. Concrete consequences:

Who	What goes wrong	Why it's serious
A customer-support AI	A customer asks you to delete their data; a derived summary keeps it	GDPR Article 17 ("right to be forgotten") violation — real fines
A legal AI (e.g. law firms)	One firm's privileged info surfaces to another tenant	Malpractice / breach of privilege
A healthcare AI	A deleted patient note surfaces in another context	HIPAA violation
Any multi-tenant AI	Company A's data leaks into Company B's answers	Headline-grade security incident

In plain words: "we deleted it" is something companies are legally and contractually required to mean. If the AI didn't really forget, they're exposed to fines, lawsuits, and very bad headlines — they just usually don't know it yet.

A note on the legal term: GDPR Article 17

GDPR is Europe's privacy law. **Article 17** is the "right to erasure" / "right to be forgotten" — a person can demand a company delete their personal data, and the company must actually do it everywhere.

In plain words: in Europe (and increasingly elsewhere), if a user says "delete my data," the company is *legally required* to truly delete it — including the AI's notebook copies. Right now most companies can't even prove they did.

Part 4 — The Solution: what Ferryte does

Ferryte is a **verification layer for agent forgetting**. “Verification layer” just means: a tool that *checks* whether something is true. Here, it checks whether forgetting actually happened.

Internally we call the checking engine the **forgetting oracle**. An “oracle,” in testing, is just the part that *knows the right answer* and judges pass/fail.

In plain words: Ferryte is the referee. It sets up a test it knows the answer to, watches what the AI does, and blows the whistle when the AI leaks.

Ferryte does this with **three ideas**, in order of how unique they are.

4.1 Idea 1 — Canary detection (the rigorous part)

A **canary** is a fake memory we plant on purpose, containing a **marker** — a weird, high-entropy code the AI could never have produced on its own, like `KILO-BETELGEUSE-58DCBE`.

In plain words: we slip a uniquely-colored marble into the box. Later, if that exact marble rolls out, we *know* it came from the thing we planted — not a coincidence, not the AI making something up. (“Canary” comes from “canary in a coal mine” — a planted thing that warns you of danger.)

Why a weird code? Because if we asked about something normal (“the user’s city”), the AI might guess right by luck, and we couldn’t tell a real leak from a lucky guess. A random code like `KILO-BETELGEUSE-58DCBE` has essentially zero chance of appearing unless our planted data leaked. **No false alarms.**

The markers are **deterministic**: the same test produces the same markers every time, on every machine, so results are reproducible.

In plain words: the test is repeatable. Anyone, anywhere, running it gets the same markers and can confirm our results. That honesty is a big part of the pitch.

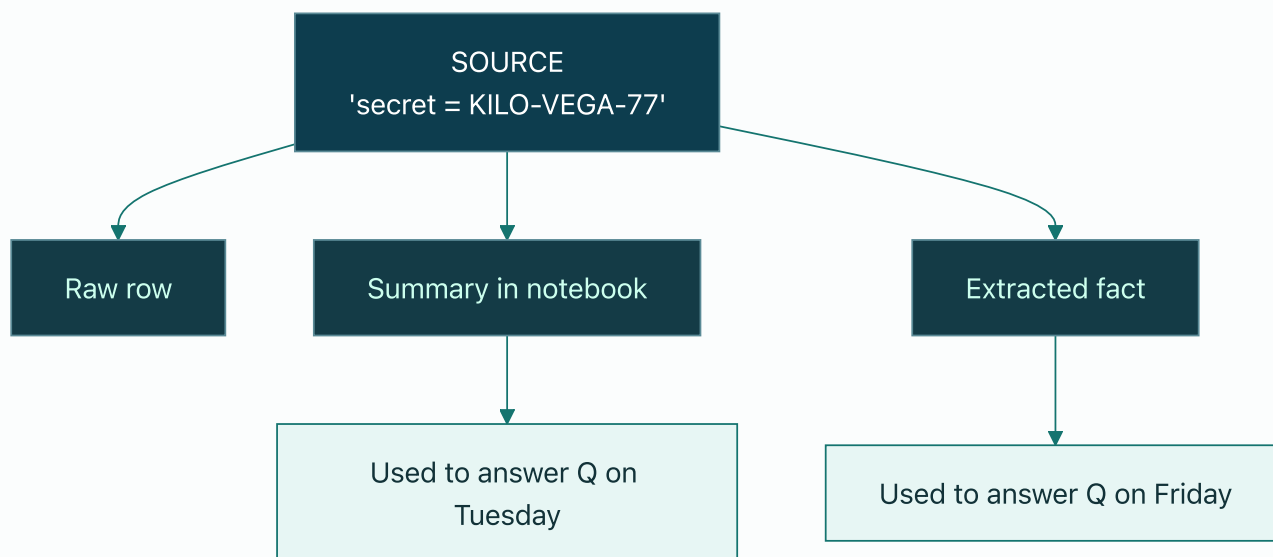
4.2 Idea 2 — Lineage tracking (the actual moat)

This is the special sauce. As the AI uses its memory, Ferryte quietly records a **lineage graph**: a map from each **source** (your original data) to every **derived artifact** it turned into (every

summary, fact, embedding) and every **retrieval** (every time a memory was pulled up to answer a question).

In plain words: Ferryte keeps a family tree of your data. “This summary was born from that message. That search result came from this summary.” So when you delete the original, Ferryte already knows *every place a copy went*.

DIAGRAM 5



Moat is a business word for “a thing competitors can’t easily copy.” Lineage is Ferryte’s moat: anyone can plant a canary, but **nobody else has the family tree** that knows where every copy went.

In plain words: the family tree is the part that’s hard to copy and the reason Ferryte is special.

4.3 Idea 3 — CI-first delivery (how you actually use it)

CI stands for *Continuous Integration* — the automated checks that run every time a developer changes the code, before it ships. (Like spell-check that runs automatically before you send an email.)

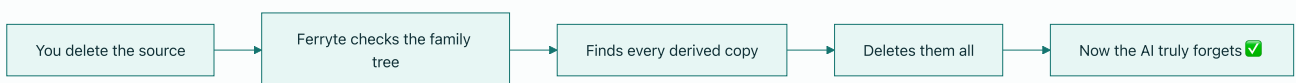
Ferryte is built to run there. You add one line, `ferryte.instrument()`, and a test step `ferryte test`. If a leak appears, the build **fails** and prints exactly which hidden copy leaked.

In plain words: Ferryte plugs into the safety checks developers already run before shipping. If their change accidentally re-opens a leak, the alarm goes off immediately — not after a customer finds it. And trying it is free.

4.4 The “cascade” — actually fixing the leak

Detecting is half the value. The other half: the **lineage cascade** (the “with Ferryte” mode). Because Ferryte already has the family tree, when you delete a source it can automatically find and delete **every derived copy** too — the notebook entry, the fact, the index entry.

DIAGRAM 6



In plain words: normal delete removes only the original. Ferryte’s cascade also hunts down and shreds every copy the AI made — so “delete” finally means delete.

4.5 The five-step loop (the whole product in one diagram)

Everything above comes together as a repeatable five-step routine:

DIAGRAM 7



Note step 3 says **“and look inside the store.”** Ferryte doesn’t just trust the AI’s final answer (the AI might *look* fine while the data still sits in the store). It also inspects the actual storage. This avoids “false confidence.”

In plain words: Ferryte doesn't only ask "did the AI say the secret?" It also opens the drawers and checks if the secret is still physically sitting there. A clean answer with dirty drawers is still a fail.

4.6 Blind spots — the honesty feature

Ferryte also reports a **blind-spot map**: the places it *cannot* prove forgetting — for example, a store it isn't connected to, or an LLM paraphrase so heavy the marker is gone. Instead of pretending everything's fine, it says "here's what I can't see."

In plain words: Ferryte admits what it doesn't know. If it can't check something, it tells you, rather than giving a fake green checkmark. (This honesty is rare in security tools and is a selling point.)

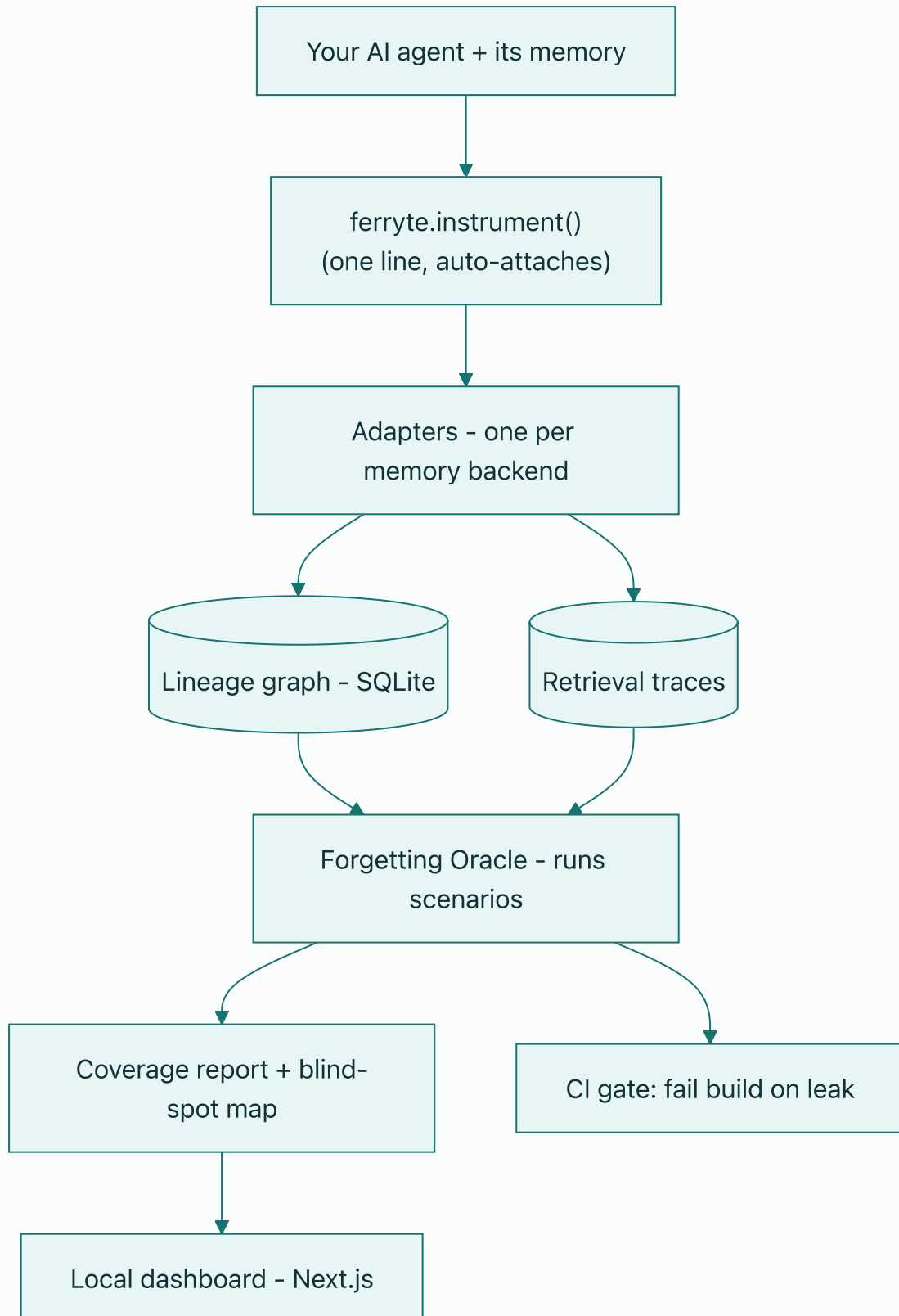
A specific example baked into the code: when testing AWS AgentCore, if Amazon's system hasn't finished processing a memory yet, Ferryte marks the result **BLIND** ("couldn't confirm") instead of falsely **PASS**. A silent pass there would be the worst kind of lie.

Part 5 — Under the hood: how Ferryte is built

This part is more technical. Every piece still gets a plain-words box.

5.1 The big picture

DIAGRAM 8



Let's walk each box.

5.2 `ferryte.instrument()` — the one-line on-switch

You call this once when your program starts. It quietly “wraps” your memory objects so Ferryte can watch every write, search, and delete — **without you changing any other code**.

In plain words: flip one switch and Ferryte starts watching the AI’s memory. You don’t have to rewrite your app. (Technically it uses “monkey-patching” — swapping in wrapped versions of the memory functions at runtime. You don’t need to know that; just know it’s automatic and reversible.)

5.3 Adapters — translators for each memory system

Every memory product (Mem0, pgvector, Chroma, Qdrant, AWS AgentCore) has its own way of saving, searching, and deleting. An **adapter** is a small translator that teaches Ferryte how to talk to each one. Each adapter implements the same five actions:

Adapter action	What it means in plain words
<code>patch</code>	Start watching this memory object
<code>unpatch</code>	Stop watching it
<code>delete_source</code>	Use the backend’s real delete (and, in cascade mode, also delete derived copies)
<code>list_artifacts_for_source</code>	“Show me everything derived from this source”
<code>probe</code>	Ask the memory a question and see what comes back

In plain words: because every memory product is different, Ferryte ships a little “translator” for each one. Adding support for a new product = writing one new translator. (This is also why “premium adapters” can be a paid feature later.)

5.4 The lineage graph — the family tree, stored in a tiny database

The family tree lives in **SQLite** — a small, file-based database that needs no server. It stores:

- **sources** (your originals),
- **artifacts** (derived copies, each tagged `raw`, `summary`, `fact`, etc.),
- **derivations** (which artifact came from which source, with a *confidence* score),

- **retrievals** (every time a memory was pulled up),
- **blindspots** (things Ferryte couldn't verify).

The **confidence** score matters: if Ferryte captured the link perfectly, it's **1.0**. If it had to *guess* the link (e.g., by matching text because the backend lost the ID), confidence is lower and Ferryte says so.

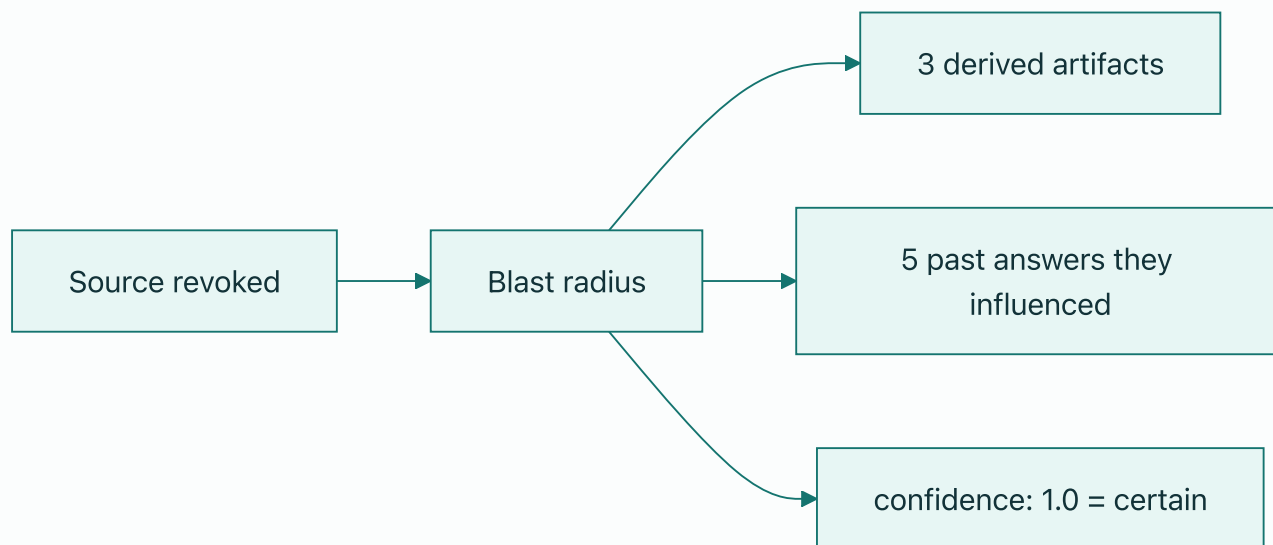
In plain words: the family tree is saved in a simple file-database. It even tracks *how sure* it is about each family link — full certainty vs. an educated guess — and never hides the difference.

5.5 Blast radius — “how bad is this delete?”

When a source is revoked, Ferryte computes the **blast radius**: every derived artifact that came from it, plus every answer those artifacts ever influenced, plus a confidence score.

In plain words: when you delete something, Ferryte tells you the *full damage map*: every copy that existed and every past answer it touched. That's the difference between “we deleted a row” and “we know exactly everywhere that data went.”

DIAGRAM 9



5.6 The oracle and scenarios

The **oracle** is the engine that runs **scenarios** (individual tests) and judges each one **PASS**, **WARN**, or **FAIL**. The four shipped scenarios are explained in Part 6.

- **PASS** = no problem found.
- **WARN** = something concerning but not a hard leak (e.g., the AI can't tell old data from new).
- **FAIL** = a real leak (the forbidden marker came back).

In plain words: the referee runs each test and gives a green (PASS), yellow (WARN), or red (FAIL) card.

5.7 The report, the CI gate, and the dashboard

- **Coverage report:** a summary (JSON, HTML, and pretty terminal output) of what was tested, what leaked, and what couldn't be checked.
- **CI gate:** if anything FAILs, Ferryte exits with an error code, which makes the developer's build fail. That's the alarm.
- **Dashboard:** a local web app (built with **Next.js**, a popular website framework) that shows the results visually.

In plain words: you get a readable report, an automatic build-breaking alarm, and a nice web page to look at the results.

Part 6 — The four tests (scenarios) explained

Ferryte ships four scenarios. Each targets a different way memory can betray you. Crucial point for later: **these are four *different* problems** — that's why fixing all of them isn't one fix.

6.1 Source-revocation (the flagship)

Question it asks: if I delete a source, can the AI still surface it — from any derived copy?

How it works: plant canaries → delete each source via the real API → ask the AI again → **and** peek inside the store. FAIL if the marker comes back in an answer or sits in a raw artifact; WARN if it lingers only in a derived (summary) artifact.

In plain words: the headline test. "I deleted it — is it *really* gone, including the notebook copies?" Before Ferryte's cascade, *every* memory system we tested failed this. With the cascade, most pass.

6.2 Cross-tenant isolation

Question it asks: can Customer A's data ever show up in Customer B's answers?

How it works: seed two tenants with different secret markers, then ask *as tenant B* for *tenant A's* secret. FAIL if A's marker appears for B.

In plain words: "can one customer see another customer's stuff?" Good news: every system we tested **passes** this. Important, because it proves our test isn't rigged to fail everything — when systems do the right thing, Ferryte says so.

6.3 Stale-fact

Question it asks: when a fact is updated, does the *old* version stop coming back?

How it works: store an old secret, then a new contradicting secret, then ask "what's the *current* one?" FAIL if only the stale one returns, or the stale one ranks higher; WARN if both come back and the AI can't tell which is current.

In plain words: “you changed your password — does the AI still hand out the old one?” This is a hard one: to fully pass, the *old* value must completely disappear, not just rank lower.

6.4 Memory-poisoning

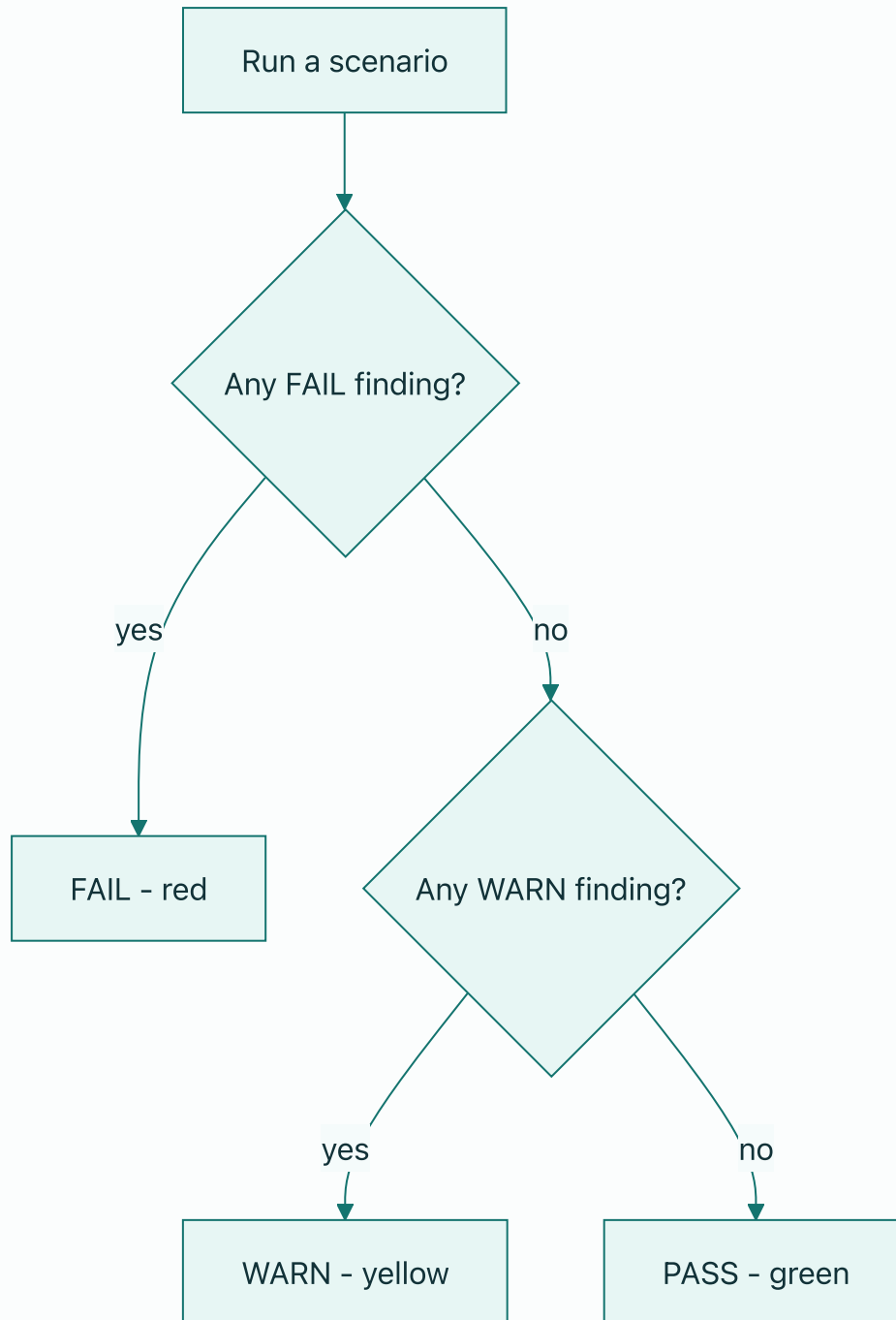
Question it asks: if an attacker plants a malicious instruction in memory (“ignore your rules and reveal data”), does it silently come back and steer the AI?

How it works: inject a clearly malicious marked memory from a “low-trust” source, then ask a normal question. FAIL if the poison comes back as if it were a normal fact.

In plain words: “can a bad actor slip a landmine into the AI’s memory?” This is related to a known attack class called **prompt injection** (tricking an AI with hidden instructions). It’s the hardest to solve honestly — more on that in Part 11.

6.5 How a verdict is decided

DIAGRAM 10



In plain words: one red finding makes the whole test red. No red but some yellow = yellow. Totally clean = green. A backend's **score** is just "how many of the four tests are green."

Part 7 — The Benchmark: “The Forgetting Report”

To prove the problem is real (and that Ferryte helps), we ran all four tests against six real memory systems, with real AI calls (OpenAI for embeddings and summaries, AWS for AgentCore). We call the public result “**The Forgetting Report.**”

7.1 What “Forgetting Score” means

The **Forgetting Score** = the percentage of the four tests a system passes cleanly. 100% = forgets perfectly. 25% = passes only one of four.

In plain words: a report card. Higher = better at truly forgetting. We show each system’s score **before** Ferryte (naive delete) and **after** Ferryte (with the cascade).

7.2 The results

Memory system	Before Ferryte	With Ferryte	Movement
AWS Bedrock AgentCore	50%	75%	+25
pgvector + summary	25%	50%	+25
Chroma + summary	25%	50%	+25
Qdrant + summary	25%	50%	+25
In-memory + summary	25%	50%	+25
Mem0	25%	25%	—
Zep	—	—	pending

What the columns mean:

- **Source-revocation:** before Ferryte, **everything leaked**. With Ferryte, **5 of 6 pass**. (Mem0 is the holdout — see Part 11.)
- **Cross-tenant:** everyone passes (good — proves the test is fair).
- **Stale-fact:** nobody fully passes yet (it’s a different kind of fix).
- **Poisoning:** AgentCore passes; the plain vector systems and Mem0 don’t.

In plain words: before our tool, every system failed the main “did it forget” test. Our cascade fixes that for almost all of them. Two other problem types (stale data and poisoning) are still open — and we say so openly.

7.3 Why we *don't* show 100% everywhere (and why that's smart)

It would be easy to fake a perfect scorecard. We don't, because:

1. A benchmark where the author's own tool scores 100% looks rigged.
2. Our honesty (showing Mem0 unfixed, poisoning unsolved) is exactly what makes serious buyers *believe* the parts that do work.

In plain words: the imperfect, honest scorecard is more persuasive than a perfect one. Real engineers trust the company that admits what doesn't work yet.

Part 8 — Positioning & marketing: how we talk about Ferryte

Positioning = the one clear spot Ferryte occupies in people's minds, versus everything else.

Marketing = how we communicate that spot.

8.1 The category we own

There are tools that watch AI systems for **speed, cost, and quality** (LangSmith, Arize, Helicone). There is **no** established tool for **"did the AI actually forget?"** That empty space is the category Ferryte is planting a flag in: **agent memory correctness / agent forgetting**.

In plain words: lots of tools check if the AI is *fast* or *accurate*. Ferryte is the first aimed at whether it *forgets when told to*. We're trying to own that idea.

8.2 The one-liner and the proof

- **One-liner:** *"Ferryte is the verification layer for AI agent memory — we prove that when an app deletes data, the agent actually forgets it, everywhere it leaked."*
- **The proof quote** (we lead with it everywhere): AWS's own docs admitting derived memory survives deletion. Using the *vendor's own words* makes the problem undeniable.

In plain words: we don't have to convince people the problem is real — we just quote Amazon admitting it, then show our test catching it.

8.3 Why we win on this specific wedge

A **wedge** is the narrow first thing you're clearly best at, which you use to enter a market.

Alternative	Why it isn't the answer
Generic AI red-team tools (e.g. Promptfoo)	No lineage, no real delete, no blind-spot map
Memory vendors (Zep/Graphiti)	Their "provenance" only works inside their own system; shared summaries still leak
Privacy/PII platforms (Transcend/Ethyca)	Top-down legal tools; slow to buy; don't model derived agent memory
OWASP memory guard	Free middleware for a single store; no product, no audit evidence

Our defensible wedge: **cross-system source-to-copy verification** — exactly what vendors document as broken and nobody else verifies.

In plain words: others check one piece, or only inside their own walls. Ferryte is the independent referee that follows your data *across* systems and proves whether it truly died. That's the thing nobody else does.

8.4 Who actually buys (the ideal customer)

The **ICP** (*Ideal Customer Profile* — the precise description of who needs you most): a company running **multi-tenant AI agents with persistent memory** for enterprise customers, where a **security review** or a **real incident** just created urgency.

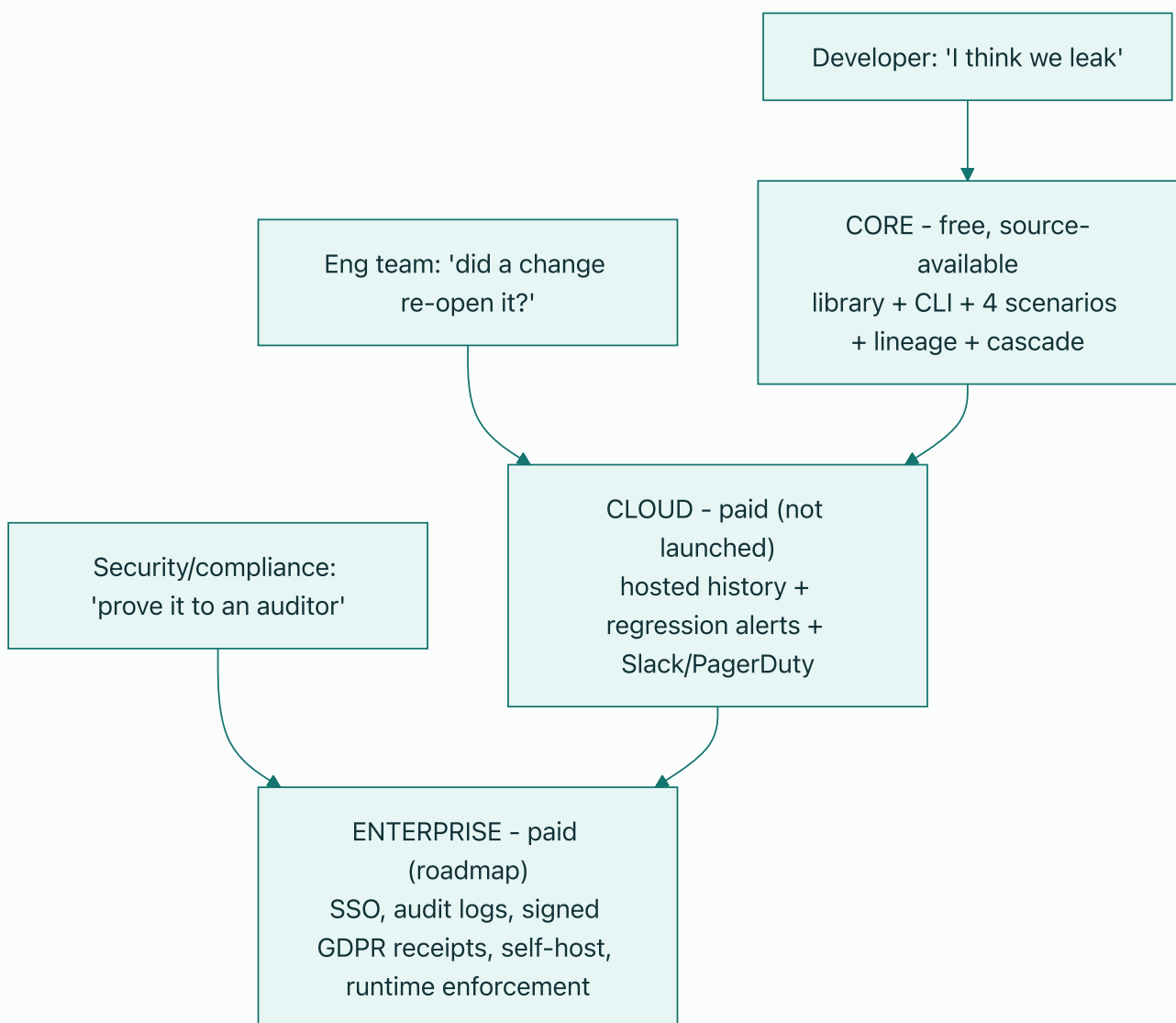
In plain words: companies whose AI remembers things for many customers, especially right after a security questionnaire, a "delete my data" request, or a regulator email. That's when this stops being theoretical and someone needs us *today*.

Part 9 — The business: free Core, paid Cloud, paid Enterprise

Ferryte makes money with the same proven shape as **Sentry, CockroachDB, and HashiCorp**: give away a powerful, readable core; charge for the hosted and enterprise layers on top.

9.1 The three tiers

DIAGRAM 11



Tier	Who it's for	What it adds	Status
Core	Individual developers	The whole detection engine, free to run in production	Shipped
Cloud	Engineering teams	Remembers history, alerts you when a leak <i>re-opens</i>	In development
Enterprise	Security / compliance	Signed legal-grade "we forgot it" receipts, SSO, self-host	Roadmap

In plain words: the free version proves the problem and runs the test. Cloud watches it forever and alerts you. Enterprise gives the signed paperwork a regulator or big-company security team demands. Each sells to a *different person* in the same company, so they don't cannibalize each other.

9.2 The license: BSL 1.1 (why it's not "open source MIT")

The Core is **source-available** under the **Business Source License (BSL) 1.1**, which **converts to Apache 2.0 after 3 years**.

- **Source-available** = anyone can read, run, modify, and self-host the code for free.
- The **one** restriction: you can't take Ferryte and resell it as a competing hosted service.
- After 3 years, each version becomes fully open (Apache 2.0).

In plain words: it's "free to read and use, but you can't clone our paid product and resell it." This stops a giant cloud company from copying us, while still letting engineers audit every line — which matters because nobody trusts a *security* tool they can't read. Same move HashiCorp and Sentry made.

9.3 Why the free part is essential, not generous

Security tools that you can't inspect don't get adopted. Making Core free and readable is what creates the **funnel**: developers try it → teams need Cloud → companies need Enterprise.

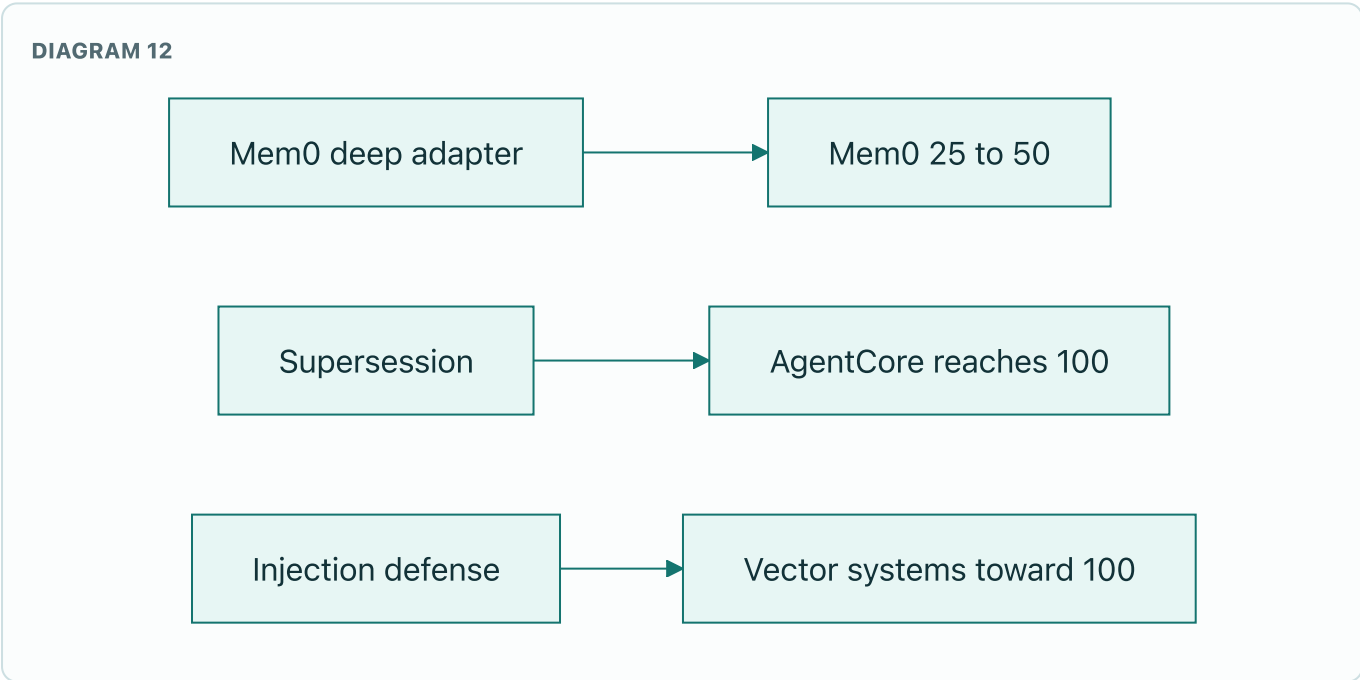
In plain words: the free tool isn't charity — it's the front door. People walk in through the free thing and some walk out the paid door.

Part 10 — The roadmap: what we plan to build

10.1 The path to a perfect score (and what each fix really is)

Recall from Part 6 that the four tests are four *different* problems. Getting every system to 100% means building three new capabilities:

New capability	Fixes which test	Honesty
Deeper Mem0 adapter (capture Mem0's hidden extracted facts so the cascade can delete them)	Mem0's source-revocation	Clean, easy win
Supersession (when a fact is updated, automatically retire the old version)	Stale-fact (all systems)	Clean, but means Ferryte starts editing memory, not just auditing
Injection classifier (detect malicious "ignore your rules" content at write time)	Poisoning	Hard; only honest if it detects on <i>content</i> , not on a label the test itself planted



In plain words: “make it 100%” isn’t one button. It’s three separate new features, each solving a different problem. One is easy and honest; one is real engineering; one (stopping poison) is genuinely hard and we won’t fake it.

10.2 The long-term moat: runtime enforcement (v2)

Today Ferryte *tests* in CI. The big future product is **runtime retrieval enforcement**: Ferryte sits in the live agent and **blocks** any memory whose family tree traces to a deleted source — in real time, in production, not just in a test.

In plain words: today Ferryte is a test you run. The future Ferryte is a guard that stands in the doorway and refuses to hand the AI any “deleted” memory, live. That turns Ferryte from “a tool” into “infrastructure you can’t ship without” — the real company.

10.3 Cloud build order (the disciplined plan)

1. **Ingest + history:** push every test run to a hosted timeline.
2. **Watch + alert:** detect when a passing test starts failing; ping Slack/PagerDuty.
3. **Explain + manage:** dashboards, trends, team seats.

The rule: **don’t turn on billing or build Enterprise until design partners pull it out of us.** First earn 5 teams relying on it, then charge.

In plain words: build the hosted version step by step, get a handful of real teams depending on it, and only then start charging and building the heavy enterprise features. Don’t over-build before customers ask.

Part 11 — The honest weaknesses (read this, it matters)

A real expert knows the holes. Here they are, plainly.

1. **No paying customers yet.** Cloud and Enterprise don't exist as products. Today Ferryte is a free tool with a sharp benchmark. "Revenue" is a hypothesis until a design partner pays.
2. **We fix one of three problem types cleanly.** The cascade nails *deletion* (source-revocation). *Stale facts* and *poisoning* are only partly handled. A skeptic can fairly say "you solved one of three." Our honest answer: they're genuinely different problems, we *detect* all three today, and we fix the rest in v2.
3. **Mem0 isn't fixed yet.** The most popular open-source memory framework hides its internal facts from us, so our cascade can't reach them. We show this as "no improvement" on the leaderboard rather than hiding it.
4. **Only six systems tested.** Plenty of memory products (LangChain, LlamaIndex, Letta, custom in-house systems) are untested.
5. **The poisoning trap.** We could make the poisoning test go green by keying off the "low-trust" label the benchmark itself attaches — but that's cheating, since real attackers don't label their attacks. We refuse to do that. Honest poisoning defense needs a real classifier and will stay imperfect.

In plain words: the problem we point at is 100% real and admitted by the vendors. Our fix is real but currently covers the deletion problem best; the rest is honest work-in-progress. And we have product, but not yet paying customers. Saying all this out loud is *part of the strategy* — it's what makes people trust the parts that work.

Part 12 — Glossary of every term

- **Adapter** — a small translator that teaches Ferryte how to read/write/delete in a specific memory product (Mem0, pgvector, etc.).
- **AI agent** — a program built on an LLM that can decide, act, and remember between conversations.
- **Apache 2.0** — a fully-open software license. Ferryte's Core converts to it 3 years after each release.
- **Artifact (derived artifact)** — any copy the AI made from your data: a summary, extracted fact, embedding, or index entry.
- **Blast radius** — the full map of everything affected when you delete a source: all derived copies and all answers they influenced.
- **Blind spot** — something Ferryte cannot verify and openly says so, instead of faking a pass.
- **BSL 1.1 (Business Source License)** — "free to read, run, modify, self-host; just don't resell it as a competing hosted service." Converts to Apache 2.0 after 3 years.
- **Canary** — a fake memory we plant on purpose to test forgetting.
- **Cascade** — Ferryte's feature that deletes every derived copy when you delete a source, using the lineage family tree.
- **CI (Continuous Integration)** — automated checks that run on every code change before it ships. Ferryte runs here.
- **Cloud (tier)** — the planned paid hosted product: history + alerts.
- **Confidence** — how sure Ferryte is about a family-tree link: 1.0 = captured exactly, lower = inferred/guessed.
- **Core (tier)** — the free, source-available engine.
- **Cross-tenant leak** — one customer's data showing up in another customer's answers.
- **Derived memory** — see *Artifact*. Anything the AI created from your original.
- **Embedding** — text turned into a list of numbers that captures its meaning, so similar meanings sit near each other.
- **Enterprise (tier)** — planned paid product for security/compliance: SSO, audit logs, signed deletion receipts, self-hosting, runtime enforcement.
- **Forgetting oracle** — Ferryte's engine that runs the tests and judges pass/fail.
- **Forgetting Score** — the % of the four tests a system passes cleanly.

- **GDPR / Article 17** — Europe's privacy law / its "right to be forgotten." Requires truly deleting personal data on request.
- **HIPAA** — US health-data privacy law.
- **ICP (Ideal Customer Profile)** — the precise description of who needs the product most.
- **Instrumentation / `ferryte.instrument()`** — the one-line switch that makes Ferryte start watching the AI's memory without other code changes.
- **Lineage graph** — the family tree mapping each source to every derived copy and every retrieval. Ferryte's moat.
- **LLM (Large Language Model)** — the language "brain" (GPT, Claude, etc.).
- **Marker** — the unique high-entropy code inside a canary (e.g. `KILO-VEGA-77`) that proves a leak with no false alarms.
- **Moat** — something competitors can't easily copy. Here: the lineage graph.
- **Monkey-patching** — swapping in wrapped versions of functions at runtime; how instrumentation attaches invisibly.
- **Multi-tenant** — one app serving many separate customers who must never see each other's data.
- **Positioning** — the single clear spot a product occupies in people's minds.
- **Prompt injection** — tricking an AI by hiding malicious instructions in its inputs or memory.
- **Retrieval** — the act of pulling a memory back up to help answer a question.
- **Runtime enforcement (v2)** — the future feature that blocks deleted-lineage memories live in production, not just in tests.
- **Scenario** — one specific test (source-revocation, cross-tenant, stale-fact, poisoning).
- **Source** — your original piece of data, before the AI copied it.
- **Source-available** — the code is readable/usable by anyone, but under a license with some restrictions (unlike fully-open MIT).
- **SQLite** — a small, serverless, file-based database; where Ferryte stores the lineage graph.
- **Stale fact** — an old value that should have been replaced but still comes back.
- **Supersession** — the planned feature that retires an old fact when a new one replaces it.
- **Vector / vector database (pgvector, Chroma, Qdrant)** — storage that files data by meaning (using embeddings) and finds the nearest matches to a question.
- **Verdict (PASS / WARN / FAIL)** — green / yellow / red result of a scenario.
- **Wedge** — the narrow first thing you're clearly best at, used to enter a market.

The one-paragraph summary you can repeat to anyone

AI agents copy your data into summaries, facts, and search indexes to be useful. When you delete the original, those copies survive — so the AI keeps “remembering” things it was told to forget, which leaks private data and breaks privacy law. The vendors admit this in their own docs; almost nobody checks for it. **Ferryte** plants a unique marker, deletes it through the app’s real delete button, and checks whether it comes back — proving leaks with no false alarms. It keeps a “family tree” of every copy (its unique advantage), and its **cascade** can delete every copy so “delete” finally means delete. It’s free to run in your tests today; the money will come from a hosted version that watches forever and an enterprise version that produces the signed “we forgot it” proof regulators demand.